

GUÍA DIDÁCTICA

Desarrollo de Aplicaciones Móviles Android

Sistema de Reserva de Canchas Deportivas

Persistencia local con SQLite + Room · Arquitectura MVVM · Java

NIVEL

Iniciación a la programación móvil — Java

Ruta de la serie de guías

- Guía 1 (esta guía): App local con SQLite + Room — sin conexión a internet.
- Guía 2 (siguiente): Sincronización en línea con Firebase Firestore.
- Guía 3 (avanzada): App conectada a API REST propia (Laravel + MySQL) con Retrofit.

Carrera de Ingeniería de Sistemas Informáticos

Programación de Dispositivos Móviles

Índice de Contenidos

Índice de Contenidos	2
Introducción	4
Competencias a desarrollar	4
Requisitos previos	4
Herramientas y tecnologías utilizadas	4
1.1 ¿Por qué SQLite y no MySQL en esta primera app?	5
1.2 ¿Qué es Room?	5
1.3 Arquitectura MVVM	5
2.1 Modelo simplificado — 6 entidades	6
2.2 Atributos y relaciones	6
Cliente	6
Cancha	6
Disciplina	6
Reserva	6
ReservaDetalle	6
Pago	6
2.3 Diagrama de relaciones	7
3.1 Crear el proyecto	8
3.2 Agregar dependencias de Room	8
3.3 Estructura de paquetes recomendada	8
4.1 Cliente	10
4.2 Cancha	10
4.3 Disciplina	11
4.4 Reserva (con clave foránea a Cliente)	11
4.5 ReservaDetalle (con 3 claves foráneas)	12
4.6 Pago	13
5.1 ClienteDao	15
5.2 CanchaDao	15
5.3 ReservaDao (con consultas relacionadas)	15
7.1 Repository	18
7.2 ViewModel	18
8.1 Layout: lista de canchas (item_cancha.xml)	20
8.2 Adapter para RecyclerView	20
8.3 Activity: listado de canchas	21
8.4 Formulario: crear nueva reserva	22
9.1 Flujo completo de la app	23
9.2 Verificar la base de datos con Database Inspector	23
9.3 Errores comunes y solución	23
Actividad 1 — Básica	24
Actividad 2 — Intermedia	24

Actividad 3 — Intermedia-Avanzada	24
Actividad 4 — Avanzada	24
Actividad 5 — Desafío	24
Glosario de términos	25
Conclusiones y siguientes pasos	26
Próximas guías de la serie	26

Introducción

Esta guía acompaña, paso a paso, la construcción de una aplicación Android nativa para la gestión de reservas de canchas deportivas, usando Java y la librería Room como capa de persistencia sobre SQLite. El proyecto es una versión simplificada del modelo relacional completo del sistema de reservas, adaptada para que un estudiante que recién se inicia en el desarrollo Android pueda completar un ciclo real de desarrollo: desde el diseño de la base de datos hasta una interfaz funcional, sin depender de un servidor remoto.

A diferencia de un backend web (como Laravel), en una aplicación Android la base de datos vive dentro del propio dispositivo del usuario. Room es la librería oficial de Jetpack que permite trabajar con SQLite de forma segura, verificada en tiempo de compilación y con una sintaxis muy cercana a la que ya conocen los estudiantes que han trabajado con Eloquent en Laravel.

Competencias a desarrollar

Al finalizar esta guía, el estudiante será capaz de:

- Diseñar un modelo de datos relacional simplificado apto para una app móvil.
- Configurar un proyecto Android con Room y sus dependencias.
- Implementar entidades, relaciones y consultas (DAO) con Room.
- Aplicar el patrón de arquitectura MVVM (Model-View-ViewModel) en Android.
- Construir una interfaz de usuario funcional con listas (RecyclerView) y formularios.
- Integrar todas las capas en un flujo completo: registrar cliente, crear reserva, ver historial.

Requisitos previos

- Conocimientos básicos de programación orientada a objetos en Java.
- Nociones de bases de datos relacionales (tablas, claves primarias y foráneas).
- Android Studio instalado (versión Iguana o superior recomendada).
- JDK 17 configurado (Android Studio lo incluye embebido).

Herramientas y tecnologías utilizadas

Herramienta	Uso en el proyecto
Java	Lenguaje principal de desarrollo
Android Studio	IDE de desarrollo
Room (Jetpack)	ORM sobre SQLite
SQLite	Motor de base de datos local embebido
Executors (java.util.concurrent)	Operaciones asíncronas (acceso a BD sin bloquear la UI)
ViewModel + LiveData	Arquitectura MVVM
RecyclerView	Listados dinámicos (canchas, reservas)
View Binding	Acceso seguro a vistas del layout

UNIDAD 1

Fundamentos: SQLite, Room y arquitectura MVVM

1.1 ¿Por qué SQLite y no MySQL en esta primera app?

SQLite es un motor de base de datos relacional embebido: no requiere un servidor independiente, guarda toda la información en un solo archivo dentro del propio dispositivo Android. Esto lo hace ideal para el primer contacto con persistencia de datos en apps móviles, ya que el estudiante no necesita configurar un backend, una API ni una conexión a internet.

Diferencia clave con lo ya visto en Laravel

- En Laravel, la base de datos (MySQL) vive en un servidor y se accede vía Eloquent a través de HTTP.
- En Android con Room, la base de datos vive en el propio celular y se accede directamente en el mismo proceso de la app, sin red.
- Room cumple, dentro de Android, un rol equivalente al de Eloquent en Laravel: es un ORM.

1.2 ¿Qué es Room?

Room es una librería de la biblioteca de Jetpack que añade una capa de abstracción sobre SQLite. Con Room se trabaja con tres piezas principales:

- Entity: una clase Java (POJO) anotada con `@Entity` que representa una tabla.
- DAO (Data Access Object): una interfaz anotada con `@Dao` que declara las consultas SQL como métodos.
- Database: una clase abstracta anotada con `@Database` que une las entidades y expone los DAO.

1.3 Arquitectura MVVM

El proyecto sigue el patrón MVVM (Model-View-ViewModel), recomendado oficialmente por Google, que separa responsabilidades en tres capas:

Capa	Responsabilidad	Elementos en este proyecto
Model	Datos y acceso a la base de datos	Entities, DAO, Database, Repository
ViewModel	Lógica de presentación, estado de la UI	ClienteViewModel, ReservaViewModel
View	Interfaz gráfica, interacción del usuario	Activities, Fragments, layouts XML

Este mismo patrón de separación en capas (modelo → lógica → presentación) es análogo al que ya se practica en el patrón MVC de Laravel, solo que adaptado al ciclo de vida de una app móvil.

UNIDAD 2

Diseño del modelo de datos simplificado

El sistema de reservas de canchas completo (visto en el diagrama entidad-relación institucional) contempla 9 tablas, incluyendo catálogos como tipos_cancha, estados_reserva y metodos_pago. Para esta primera aplicación Android, esas tablas de catálogo se simplifican como simples campos de texto, conservando únicamente las 6 entidades esenciales que permiten practicar relaciones reales (uno a muchos) con Room.

2.1 Modelo simplificado — 6 entidades

Entidad	Rol en el sistema
Cliente	Persona que realiza reservas
Cancha	Espacio deportivo disponible para reservar
Disciplina	Deporte que se juega en la cancha reservada
Reserva	Solicitud general de un cliente (puede incluir varias canchas)
ReservaDetalle	Cada cancha/horario específico dentro de una reserva
Pago	Transacción realizada por cada detalle de reserva

2.2 Atributos y relaciones

Cliente

- idCliente (PK, autogenerado)
- nombre, apellido, email, telefono

Cancha

- idCancha (PK, autogenerado)
- nombre, tipo (texto: "Fútbol 5", "Fútbol 7", "Vóley", etc.), precioBase, estado

Disciplina

- idDisciplina (PK, autogenerado)
- nombre, descripcion

Reserva

- idReserva (PK, autogenerado)
- idCliente (FK → Cliente)
- fechaReserva, estado (texto: Pendiente, Confirmada, Cancelada, Finalizada), totalReserva

ReservaDetalle

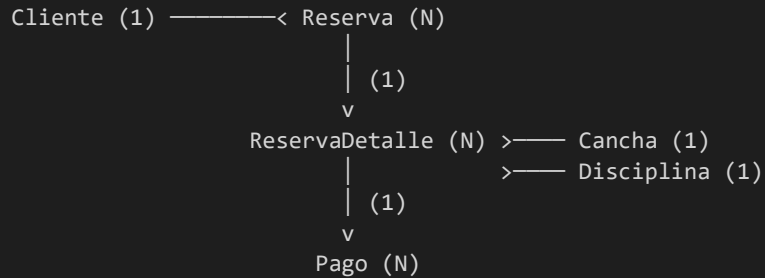
- idReservaDetalle (PK, autogenerado)
- idReserva (FK → Reserva), idCancha (FK → Cancha), idDisciplina (FK → Disciplina)
- fechaJuego, horaInicio, horaFin, precioUnitario, subtotal

Pago

- idPago (PK, autogenerado)
- idReservaDetalle (FK → ReservaDetalle)
- monto, metodoPago (texto), estado, fechaPago

2.3 Diagrama de relaciones

modelo-relaciones.txt



Regla de negocio que conserva el modelo simplificado

- Un cliente puede tener muchas reservas.
- Una reserva puede incluir varios detalles (varias canchas/horarios distintos).
- Cada detalle de reserva pertenece a una cancha y una disciplina específicas.
- El pago se realiza por cada detalle de reserva, tal como en el modelo institucional original.

UNIDAD 3

Configuración del proyecto Android

3.1 Crear el proyecto

1. Abrir Android Studio → New Project → Empty Views Activity.
2. Nombre: ReservaCanchasApp. Lenguaje: Java. Minimum SDK: API 26 (Android 8.0).
3. Finalizar y esperar la sincronización de Gradle.

3.2 Agregar dependencias de Room

Abrir el archivo build.gradle (Module :app) y agregar las siguientes dependencias. En un proyecto Java, Room usa annotationProcessor (en lugar de kapt, que es exclusivo de Kotlin) para generar el código SQL en tiempo de compilación:

```
build.gradle

// build.gradle (Module :app)
plugins {
    id 'com.android.application'
}

android {
    buildFeatures {
        viewBinding true
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_17
        targetCompatibility JavaVersion.VERSION_17
    }
}

dependencies {
    def room_version = "2.6.1"

    implementation "androidx.room:room-runtime:$room_version"
    annotationProcessor "androidx.room:room-compiler:$room_version"

    implementation "androidx.lifecycle:lifecycle-viewmodel:2.7.0"
    implementation "androidx.lifecycle:lifecycle-livedata:2.7.0"

    implementation "androidx.recyclerview:recyclerview:1.3.2"
    implementation "androidx.cardview:cardview:1.0.0"
}
```

Nota

- annotationProcessor es el mecanismo estándar de Java para generar código en tiempo de compilación: Room lo usa para crear automáticamente todo el SQL a partir de las anotaciones @Entity, @Dao y @Database.
- Tras editar el archivo, presionar "Sync Now" en la barra que aparece en Android Studio.

3.3 Estructura de paquetes recomendada

estructura de carpetas

```
com.udabol.reservacanchas/  
├── data/  
│   ├── entity/      (Cliente, Cancha, Disciplina, Reserva, ReservaDetalle, Pago)  
│   ├── dao/         (ClienteDao, CanchaDao, ReservaDao, ...)  
│   ├── AppDatabase.java  
│   └── repository/  (ClienteRepository, ReservaRepository)  
├── ui/  
│   ├── clientes/    (ClienteListActivity, ClienteAdapter)  
│   ├── canchas/     (CanchaListActivity, CanchaAdapter)  
│   └── reservas/    (NuevaReservaActivity, ReservaListActivity)  
└── viewmodel/       (ClienteViewModel, ReservaViewModel)
```

UNIDAD 4

Implementación de las Entidades (@Entity)

Cada Entity es una clase Java (un POJO — Plain Old Java Object) anotada con @Entity, que Room convierte en una tabla SQLite. Los atributos se convierten en columnas, @PrimaryKey identifica la clave primaria, y se necesitan constructor, getters y setters para que Room pueda leer y escribir cada campo.

4.1 Cliente

Cliente.java

```
@Entity(tableName = "clientes")
public class Cliente {

    @PrimaryKey(autoGenerate = true)
    private int idCliente;
    private String nombre;
    private String apellido;
    private String email;
    private String telefono;

    public Cliente(String nombre, String apellido,
                   String email, String telefono) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
    }

    public int getIdCliente() { return idCliente; }
    public void setIdCliente(int idCliente) { this.idCliente = idCliente; }
    public String getNombre() { return nombre; }
    public String getApellido() { return apellido; }
    public String getEmail() { return email; }
    public String getTelefono() { return telefono; }
}
```

4.2 Cancha

Cancha.java

```
@Entity(tableName = "canchas")
public class Cancha {

    @PrimaryKey(autoGenerate = true)
    private int idCancha;
    private String nombre;
    private String tipo;           // "Fútbol 5", "Fútbol 7", "Vóley"...
    private double precioBase;
    private String estado = "Activa";

    public Cancha(String nombre, String tipo, double precioBase) {
        this.nombre = nombre;
        this.tipo = tipo;
        this.precioBase = precioBase;
    }
}
```

```
}

public int getIdCancha() { return idCancha; }
public void setIdCancha(int idCancha) { this.idCancha = idCancha; }
public String getNombre() { return nombre; }
public String getTipo() { return tipo; }
public double getPrecioBase() { return precioBase; }
public String getEstado() { return estado; }
public void setEstado(String estado) { this.estado = estado; }
}
```

4.3 Disciplina

Disciplina.java

```
@Entity(tableName = "disciplinas")
public class Disciplina {

    @PrimaryKey(autoGenerate = true)
    private int idDisciplina;
    private String nombre;
    private String descripcion;

    public Disciplina(String nombre, String descripcion) {
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    public int getIdDisciplina() { return idDisciplina; }
    public void setIdDisciplina(int id) { this.idDisciplina = id; }
    public String getNombre() { return nombre; }
    public String getDescripcion() { return descripcion; }
}
```

4.4 Reserva (con clave foránea a Cliente)

Reserva.java

```
@Entity(
    tableName = "reservas",
    foreignKeys = @ForeignKey(
        entity = Cliente.class,
        parentColumns = "idCliente",
        childColumns = "idCliente",
        onDelete = ForeignKey.CASCADE
    ),
    indices = { @Index("idCliente") }
)
public class Reserva {

    @PrimaryKey(autoGenerate = true)
    private int idReserva;
    private int idCliente;
    private String fechaReserva;
    private String estado = "Pendiente";
    private double totalReserva;

    public Reserva(int idCliente, String fechaReserva, double totalReserva) {
```

```

        this.idCliente = idCliente;
        this.fechaReserva = fechaReserva;
        this.totalReserva = totalReserva;
    }

    public int getIdReserva() { return idReserva; }
    public void setIdReserva(int idReserva) { this.idReserva = idReserva; }
    public int getIdCliente() { return idCliente; }
    public String getFechaReserva() { return fechaReserva; }
    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }
    public double getTotalReserva() { return totalReserva; }
}

```

4.5 ReservaDetalle (con 3 claves foráneas)

ReservaDetalle.java

```

@Entity(
    tableName = "reserva_detalle",
    foreignKeys = {
        @ForeignKey(entity = Reserva.class, parentColumns = "idReserva",
            childColumns = "idReserva", onDelete = ForeignKey.CASCADE),
        @ForeignKey(entity = Cancha.class, parentColumns = "idCancha",
            childColumns = "idCancha"),
        @ForeignKey(entity = Disciplina.class, parentColumns = "idDisciplina",
            childColumns = "idDisciplina")
    },
    indices = { @Index("idReserva"), @Index("idCancha"),
        @Index("idDisciplina") }
)
public class ReservaDetalle {

    @PrimaryKey(autoGenerate = true)
    private int idReservaDetalle;
    private int idReserva;
    private int idCancha;
    private int idDisciplina;
    private String fechaJuego;
    private String horaInicio;
    private String horaFin;
    private double precioUnitario;
    private double subtotal;

    public ReservaDetalle(int idReserva, int idCancha, int idDisciplina,
        String fechaJuego, String horaInicio, String horaFin,
        double precioUnitario, double subtotal) {
        this.idReserva = idReserva;
        this.idCancha = idCancha;
        this.idDisciplina = idDisciplina;
        this.fechaJuego = fechaJuego;
        this.horaInicio = horaInicio;
        this.horaFin = horaFin;
        this.precioUnitario = precioUnitario;
        this.subtotal = subtotal;
    }

    public int getIdReservaDetalle() { return idReservaDetalle; }
    public void setIdReservaDetalle(int id) { this.idReservaDetalle = id; }
    public int getIdReserva() { return idReserva; }
}

```

```

    public void setIdReserva(int idReserva) { this.idReserva = idReserva; }
    public int getIdCancha() { return idCancha; }
    public int getIdDisciplina() { return idDisciplina; }
    public String getFechaJuego() { return fechaJuego; }
    public String getHoraInicio() { return horaInicio; }
    public String getHoraFin() { return horaFin; }
    public double getPrecioUnitario() { return precioUnitario; }
    public double getSubtotal() { return subtotal; }
}

```

4.6 Pago

Pago.java

```

@Entity(
    tableName = "pagos",
    foreignKeys = @ForeignKey(
        entity = ReservaDetalle.class,
        parentColumns = "idReservaDetalle",
        childColumns = "idReservaDetalle",
        onDelete = ForeignKey.CASCADE
    ),
    indices = { @Index("idReservaDetalle") }
)
public class Pago {

    @PrimaryKey(autoGenerate = true)
    private int idPago;
    private int idReservaDetalle;
    private double monto;
    private String metodoPago;
    private String estado = "Pendiente";
    private String fechaPago;

    public Pago(int idReservaDetalle, double monto,
        String metodoPago, String fechaPago) {
        this.idReservaDetalle = idReservaDetalle;
        this.monto = monto;
        this.metodoPago = metodoPago;
        this.fechaPago = fechaPago;
    }

    public int getIdPago() { return idPago; }
    public void setIdPago(int idPago) { this.idPago = idPago; }
    public int getIdReservaDetalle() { return idReservaDetalle; }
    public double getMonto() { return monto; }
    public String getMetodoPago() { return metodoPago; }
    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }
    public String getFechaPago() { return fechaPago; }
}

```

Buenas prácticas aplicadas

- `autoGenerate = true` delega en SQLite la generación de IDs, igual que un `AUTO_INCREMENT` en MySQL.
- `onDelete = ForeignKey.CASCADE` asegura que al borrar una reserva se borren también sus detalles, evitando registros huérfanos.
- Los índices (`@Index`) sobre las columnas FK mejoran el rendimiento de los JOIN.

- Room requiere getters públicos para leer cada campo; si el nombre del getter no sigue el patrón estándar (getNombre() para nombre), se debe anotar el campo con @ColumnInfo.

UNIDAD 5

Data Access Objects (DAO)

Un DAO es una interfaz donde cada método representa una consulta SQL. Room genera automáticamente la implementación a partir de las anotaciones `@Insert`, `@Update`, `@Delete` y `@Query`. En Java estos métodos son síncronos (no existe el `suspend` de Kotlin), por lo que siempre deben invocarse desde un hilo en segundo plano — nunca desde el hilo principal — algo que resolveremos con un `Executor` en la Unidad 6.

5.1 ClienteDao

ClienteDao.java

```
@Dao
public interface ClienteDao {

    @Insert
    long insertar(Cliente cliente);

    @Update
    void actualizar(Cliente cliente);

    @Delete
    void eliminar(Cliente cliente);

    @Query("SELECT * FROM clientes ORDER BY apellido ASC")
    LiveData<List<Cliente>> obtenerTodos();

    @Query("SELECT * FROM clientes WHERE idCliente = :id")
    Cliente obtenerPorId(int id);

    @Query("SELECT * FROM clientes WHERE email = :email LIMIT 1")
    Cliente buscarPorEmail(String email);
}
```

5.2 CanchaDao

CanchaDao.java

```
@Dao
public interface CanchaDao {

    @Insert
    long insertar(Cancha cancha);

    @Query("SELECT * FROM canchas WHERE estado = 'Activa' ORDER BY nombre")
    LiveData<List<Cancha>> obtenerActivas();

    @Query("SELECT * FROM canchas WHERE idCancha = :id")
    Cancha obtenerPorId(int id);
}
```

5.3 ReservaDao (con consultas relacionadas)

Este DAO ilustra dos técnicas importantes: `@Transaction` + `@Relation` para traer una reserva junto con sus detalles en una sola llamada, y `@Query` con JOIN para reportes.

ReservaDao.java

```
public class ReservaConDetalles {
    @Embedded
    public Reserva reserva;

    @Relation(
        parentColumn = "idReserva",
        entityColumn = "idReserva"
    )
    public List<ReservaDetalle> detalles;
}

@Dao
public interface ReservaDao {

    @Insert
    long insertarReserva(Reserva reserva);

    @Insert
    long insertarDetalle(ReservaDetalle detalle);

    @Query("SELECT * FROM reservas WHERE idCliente = :idCliente " +
        "ORDER BY fechaReserva DESC")
    LiveData<List<Reserva>> obtenerPorCliente(int idCliente);

    @Transaction
    @Query("SELECT * FROM reservas WHERE idReserva = :idReserva")
    ReservaConDetalles obtenerConDetalles(int idReserva);

    @Query("SELECT rd.*, c.nombre AS nombreCancha, " +
        "d.nombre AS nombreDisciplina " +
        "FROM reserva_detalle rd " +
        "INNER JOIN canchas c ON rd.idCancha = c.idCancha " +
        "INNER JOIN disciplinas d ON rd.idDisciplina = d.idDisciplina " +
        "WHERE rd.idReserva = :idReserva")
    List<DetalleConNombres> obtenerDetalleConNombres(int idReserva);

    @Query("UPDATE reservas SET estado = :nuevoEstado WHERE idReserva = :id")
    void cambiarEstado(int id, String nuevoEstado);
}
```

Comparación con Eloquent

- `@Relation` en Room equivale a un `hasMany()` en Eloquent: trae la reserva junto con sus detalles en un solo objeto.
- `@Transaction` garantiza que la lectura de la reserva y sus detalles ocurra de forma atómica.
- Al no existir `suspend` en Java, cada método de escritura/lectura directa (no LiveData) debe ejecutarse manualmente en un hilo en segundo plano mediante un `Executor`, para no bloquear la interfaz.

UNIDAD 6

Clase AppDatabase (instancia Singleton)

La clase `@Database` centraliza todas las entidades y expone los DAO. Se implementa como Singleton para asegurar que exista una única instancia de la base de datos durante toda la vida de la app. Como en Java no hay corrutinas, esta misma clase es un buen lugar para declarar el Executor (grupo de hilos) que usará toda la app para acceder a Room en segundo plano.

AppDatabase.java

```
@Database(
    entities = {Cliente.class, Cancha.class, Disciplina.class,
                Reserva.class, ReservaDetalle.class, Pago.class},
    version = 1,
    exportSchema = false
)
public abstract class AppDatabase extends RoomDatabase {

    public abstract ClienteDao clienteDao();
    public abstract CanchaDao canchaDao();
    public abstract DisciplinaDao disciplinaDao();
    public abstract ReservaDao reservaDao();
    public abstract PagoDao pagoDao();

    private static volatile AppDatabase INSTANCE;

    // Pool de hilos compartido para acceso a datos
    public static final ExecutorService databaseWriteExecutor =
        Executors.newFixedThreadPool(4);

    public static AppDatabase getDatabase(final Context context) {
        if (INSTANCE == null) {
            synchronized (AppDatabase.class) {
                if (INSTANCE == null) {
                    INSTANCE = Room.databaseBuilder(
                        context.getApplicationContext(),
                        AppDatabase.class,
                        "reserva_canchas_db"
                    ).build();
                }
            }
        }
        return INSTANCE;
    }
}
```

¿Por qué Singleton? ¿Por qué un Executor?

- Abrir varias instancias de la misma base de datos SQLite simultáneamente es costoso e innecesario; el patrón Singleton asegura que toda la app comparta una única conexión.
- Room prohíbe, por defecto, ejecutar consultas en el hilo principal (lanza una excepción) para evitar que la interfaz se congele.
- `ExecutorService` es la forma estándar en Java de correr tareas en segundo plano; se usa en el Repository para envolver cada llamada al DAO.

UNIDAD 7

Repository y ViewModel (arquitectura MVVM)

7.1 Repository

El Repository es una capa intermedia entre el DAO y el ViewModel. Aísla al ViewModel de los detalles de Room y facilita, más adelante, agregar otra fuente de datos (por ejemplo una API remota) sin modificar la interfaz de usuario. En Java, el Repository es también el lugar natural para enviar cada operación de escritura al Executor de AppDatabase.

ReservaRepository.java

```
public class ReservaRepository {

    private final ReservaDao reservaDao;

    public ReservaRepository(Application application) {
        AppDatabase db = AppDatabase.getDatabase(application);
        this.reservaDao = db.reservaDao();
    }

    public LiveData<List<Reserva>> obtenerReservasPorCliente(int idCliente) {
        return reservaDao.obtenerPorCliente(idCliente);
    }

    // Callback para devolver el ID generado
    public interface OnReservaCreadaListener {
        void onCreada(long idReserva);
    }

    public void crearReserva(Reserva reserva, List<ReservaDetalle> detalles,
        OnReservaCreadaListener listener) {
        AppDatabase.databaseWriteExecutor.execute(() -> {
            long idReserva = reservaDao.insertarReserva(reserva);
            for (ReservaDetalle detalle : detalles) {
                detalle.setIdReserva((int) idReserva);
                reservaDao.insertarDetalle(detalle);
            }
            listener.onCreada(idReserva);
        });
    }
}
```

7.2 ViewModel

El ViewModel expone los datos a la interfaz mediante LiveData. Como en Java no existe viewModelScope ni corrutinas, las operaciones de escritura se delegan al Repository, que a su vez las ejecuta en el Executor — el ViewModel solo recibe el resultado mediante un callback y actualiza la UI de forma segura en el hilo principal.

ReservaViewModel.java

```
public class ReservaViewModel extends AndroidViewModel {

    private final ReservaRepository repository;
```

```
public final MutableLiveData<Long> reservaCreadaId
    = new MutableLiveData<>();

public ReservaViewModel(@NonNull Application application) {
    super(application);
    repository = new ReservaRepository(application);
}

public LiveData<List<Reserva>> obtenerReservasPorCliente(int idCliente) {
    return repository.obtenerReservasPorCliente(idCliente);
}

public void crearReserva(Reserva reserva, List<ReservaDetalle> detalles) {
    repository.crearReserva(reserva, detalles, idReserva -> {
        // postValue() actualiza LiveData desde 2do plano
        reservaCreadaId.postValue(idReserva);
    });
}
}
```

Nota sobre postValue() vs. setValue()

- setValue() solo puede llamarse desde el hilo principal.
- postValue() es la versión segura para actualizar un LiveData desde un hilo en segundo plano (como el Executor), tal como se usa aquí tras crear la reserva.

UNIDAD 8

Interfaz de usuario: listados y formularios

8.1 Layout: lista de canchas (item_cancha.xml)

```
res/layout/item_cancha.xml

<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="12dp"
    app:cardElevation="3dp">

    <LinearLayout
        android:orientation="vertical"
        android:padding="16dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <TextView android:id="@+id/tvNombreCancha"
            android:textStyle="bold" android:textSize="18sp"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>

        <TextView android:id="@+id/tvTipoCancha"
            android:textColor="#2874A6"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>

        <TextView android:id="@+id/tvPrecioCancha"
            android:textColor="#E8871E" android:textStyle="bold"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>
    </LinearLayout>
</androidx.cardview.widget.CardView>
```

8.2 Adapter para RecyclerView

```
CanchaAdapter.java

public class CanchaAdapter
    extends RecyclerView.Adapter<CanchaAdapter.CanchaViewHolder> {

    public interface OnCanchaClickListener {
        void onClick(Cancha cancha);
    }

    private List<Cancha> canchas;
    private final OnCanchaClickListener listener;

    public CanchaAdapter(List<Cancha> canchas, OnCanchaClickListener listener) {
        this.canchas = canchas;
        this.listener = listener;
    }
}
```

```

static class CanchaViewHolder extends RecyclerView.ViewHolder {
    ItemCanchaBinding binding;
    CanchaViewHolder(ItemCanchaBinding binding) {
        super(binding.getRoot());
        this.binding = binding;
    }
}

@NonNull
@Override
public CanchaViewHolder onCreateViewHolder(
    @NonNull ViewGroup parent, int viewType) {
    ItemCanchaBinding binding = ItemCanchaBinding.inflate(
        LayoutInflater.from(parent.getContext()), parent, false);
    return new CanchaViewHolder(binding);
}

@Override
public void onBindViewHolder(@NonNull CanchaViewHolder holder,
    int position) {
    Cancha cancha = canchas.get(position);
    holder.binding.tvNombreCancha.setText(cancha.getNombre());
    holder.binding.tvTipoCancha.setText(cancha.getTipo());
    holder.binding.tvPrecioCancha.setText("Bs. " + cancha.getPrecioBase());
    holder.itemView.setOnClickListener(v -> listener.onClick(cancha));
}

@Override
public int getItemCount() { return canchas.size(); }

public void actualizarLista(List<Cancha> nuevaLista) {
    this.canchas = nuevaLista;
    notifyDataSetChanged();
}
}

```

8.3 Activity: listado de canchas

CanchaListActivity.java

```

public class CanchaListActivity extends AppCompatActivity {

    private ActivityCanchaListBinding binding;
    private CanchaViewModel viewModel;
    private CanchaAdapter adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        binding = ActivityCanchaListBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        viewModel = new ViewModelProvider(this).get(CanchaViewModel.class);

        adapter = new CanchaAdapter(new ArrayList<>(), cancha -> {
            Intent intent = new Intent(this, NuevaReservaActivity.class);
            intent.putExtra("idCancha", cancha.getIdCancha());
            startActivity(intent);
        });
    }
}

```

```

    });
    binding.rvCanchas.setLayoutManager(new LinearLayoutManager(this));
    binding.rvCanchas.setAdapter(adapter);

    viewModel.getCanchas().observe(this,
        lista -> adapter.actualizarLista(lista));
}
}

```

8.4 Formulario: crear nueva reserva

El flujo del formulario reproduce, en Android, el mismo flujo del diagrama institucional: el cliente selecciona cancha, disciplina, fecha y horario; la app calcula el subtotal y guarda la Reserva junto con su ReservaDetalle. El resultado se recibe observando el LiveData reservaCreadaId del ViewModel.

NuevaReservaActivity.java (fragmento)

```

binding.btnGuardarReserva.setOnClickListener(v -> {
    double precioUnitario = canchaSeleccionada.getPrecioBase();
    double subtotal = precioUnitario; // 1 hora = 1 tarifa base

    Reserva reserva = new Reserva(
        clienteActual.getIdCliente(),
        obtenerFechaHoy(),
        subtotal
    );

    ReservaDetalle detalle = new ReservaDetalle(
        0, // idReserva se completa dentro del Repository
        canchaSeleccionada.getIdCancha(),
        disciplinaSeleccionada.getIdDisciplina(),
        binding.etFecha.getText().toString(),
        binding.etHoraInicio.getText().toString(),
        binding.etHoraFin.getText().toString(),
        precioUnitario,
        subtotal
    );

    List<ReservaDetalle> detalles = new ArrayList<>();
    detalles.add(detalle);
    viewModel.crearReserva(reserva, detalles);
});

// En onCreate(), observando el resultado:
viewModel.reservaCreadaId.observe(this, idGenerado -> {
    Toast.makeText(this, "Reserva #" + idGenerado + " confirmada",
        Toast.LENGTH_LONG).show();
    finish();
});

```

UNIDAD 9

Integración final y pruebas

9.1 Flujo completo de la app

Paso	Pantalla	Acción
1	SplashActivity / MainActivity	Inicializa AppDatabase (Room)
2	ClienteFormActivity	Registra o selecciona un cliente
3	CanchaListActivity	Lista canchas activas (RecyclerView)
4	NuevaReservaActivity	Selecciona disciplina, fecha y horario
5	ReservaViewModel	Guarda Reserva + ReservaDetalle vía Repository
6	ReservaListActivity	Muestra historial de reservas del cliente
7	PagoActivity	Registra el pago del detalle de reserva

9.2 Verificar la base de datos con Database Inspector

- Ejecutar la app en un emulador o dispositivo conectado.
- En Android Studio: View → Tool Windows → App Inspection → Database Inspector.
- Seleccionar reserva_canchas_db y explorar las tablas en tiempo real, incluso ejecutar consultas SQL manuales de verificación.

9.3 Errores comunes y solución

Error	Causa probable	Solución
"Room cannot verify the data integrity"	Se cambió el esquema sin actualizar version	Incrementar version o usar <code>fallbackToDestructiveMigration()</code>
App se congela al guardar	Acceso a Room desde el hilo principal	Usar <code>AppDatabase.databaseWriteExecutor.execute { ... }</code>
FOREIGN KEY constraint failed	Se intenta insertar un detalle con idCancha inexistente	Verificar que la cancha exista antes de insertar el detalle
NullPointerException en binding	Se accede a binding tras destruir la vista	Liberar binding en <code>onDestroyView (Fragments)</code>

ACTIVIDADES PRÁCTICAS

Evaluación por competencias

Actividad 1 — Básica

Crear las 6 entidades y la clase AppDatabase. Verificar, usando Database Inspector, que las 6 tablas se generan correctamente con sus claves foráneas.

Actividad 2 — Intermedia

Implementar el CRUD completo (crear, leer, actualizar, eliminar) para la entidad Cliente, con su propia pantalla de listado y formulario.

Actividad 3 — Intermedia-Avanzada

Implementar el flujo completo de creación de una reserva: selección de cancha y disciplina, cálculo de subtotal, y guardado de Reserva junto con su ReservaDetalle.

Actividad 4 — Avanzada

Agregar la pantalla de historial de reservas por cliente utilizando @Relation para mostrar, en una sola lista, la reserva junto con todos sus detalles (canchas reservadas).

Actividad 5 — Desafío

Implementar el registro de pagos, actualizando el estado de la reserva a "Confirmada" únicamente cuando todos sus detalles tengan un pago con estado "Pagado".

Criterios de evaluación sugeridos

- Correcta definición de entidades y relaciones (25%).
- Funcionamiento del flujo de creación de reservas (30%).
- Separación de capas siguiendo MVVM (20%).
- Manejo correcto de operaciones asíncronas con Executors (15%).
- Calidad de la interfaz de usuario (10%).

Glosario de términos

Término	Definición
Room	Librería de Jetpack que actúa como capa de abstracción sobre SQLite.
Entity	Clase Java (POJO) anotada con @Entity que Room convierte en una tabla.
DAO	Interfaz que declara las operaciones de acceso a datos (consultas SQL).
LiveData	Contenedor de datos observable, consciente del ciclo de vida de la app.
ViewModel	Componente que retiene y expone datos a la UI, sobreviviendo a cambios de configuración.
Executor / ExecutorService	Mecanismo de Java para ejecutar código en un hilo en segundo plano sin bloquear la interfaz.
MVVM	Patrón de arquitectura: Model - View - ViewModel.
RecyclerView	Componente de UI para mostrar listas grandes de forma eficiente.
Singleton	Patrón de diseño que garantiza una única instancia de una clase en toda la app.

Conclusiones y siguientes pasos

Al completar esta guía, el estudiante habrá construido una aplicación Android funcional con persistencia local, replicando —de forma simplificada— el modelo relacional institucional del sistema de reservas de canchas. El dominio de Room, MVVM y el manejo de hilos en segundo plano con Executors en Java sienta la base necesaria para abordar escenarios más complejos de conectividad.

Próximas guías de la serie

Guía	Enfoque	Qué se agrega respecto a esta guía
Guía 2	Firebase Firestore	Sincronización en tiempo real entre dispositivos, sin backend propio
Guía 3	Laravel API + MySQL + Retrofit	Backend propio, autenticación con tokens, consumo de API REST desde Android

Puente con el curso de Laravel

- La Guía 3 reutilizará el mismo modelo relacional completo (9 tablas) ya trabajado en el curso de Laravel.
- Los estudiantes expondrán ese modelo como API REST (Laravel) y la consumirán desde esta misma app Android usando Retrofit, cerrando el ciclo cliente-servidor.